© Guven

Word Embeddings

Introduction

```
"You shall know a word by the company it
keeps." J.R. Firth
```

Word embedding is a collective term for **unsupervised** machine learning models that learn to map a set of words w in a vocabulary (or phrases, stems, and lemmas) to vectors of numerical values. This approach reduces the number of dimensions from the number of unique words V (i.e., vocabulary) to a much lower value N where dimensions are shared by all words, such that vectors w' are not orthogonal anymore. In addition, the way embeddings are computed, the ML models discover patterns in the word relations (such as given a context).

🔶 Practical Advantage in Text Processing and Machine Learning

After extracting words or tokens from a variable length string, such as in typical text, the preprocessing can be handled practically much easier if we are able to convert the sequence of tokens into a **fixed-size vector**. Why?

Given a word vocabulary $W = \{w_i\}$, $\|\texttt{Vocabulary}\| = V$, each word is represented by a unit vector (e.g. *one-hot encoding* in ML) such as

$$w_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, w_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, w_N = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}, W \in \mathbb{R}^{V \times V}$$

Then the word embedding maps this representation w_i, $i = 1, 2, \ldots, V$ to another representation (through **Skip-gram** or **CBOW** modeling) where the vectors are in a smaller dimension N, and $N \ll V$, and vectors w'_i, $i = 1, 2, \ldots, V$, with $\|W'\| = N$, and $W' \in \mathbb{R}^{N \times V}$

Both Skip-Gram and Continuous Bag of Words (CBOW) models use a **neural network architecture** to model the word mapping from the original W to the embedding W'

Word representation represents the word in a vector space W' so that those words are related if the word vectors are close to one another.

Recall that the above representation of words is also used in the Tf-Idf features matrix, where each column is a word, and each row is a document or a sentence.

N-Gram

An n-gram is a **contiguous** sequence of n items from a given text sample. According to the application, the items can be letters, syllables, or words. The n-grams are typically collected from a text or speech corpus. When the items are words, n-grams are also called **shingles**.

Text similarity:

- k-shingling as a similarity measure, counting k-shingles from documents
- Jaccard similarity, $J(set_1, set_2) = \dfrac{\|set_1 \cap set_2\|}{\|set_1 \cup set_2\|}$
- MinHash, a fast approximation to the Jaccard similarity
- Locality sensitive hashing

Context

Define the **context** referring to a symmetric window centered on the target word w_t, containing the surrounding tokens at a distance less than some window size $\texttt{ws}$:

$$C_t = \{w_k | k \in [t - \texttt{ws}, t + \texttt{ws}]\} \qquad (1)$$

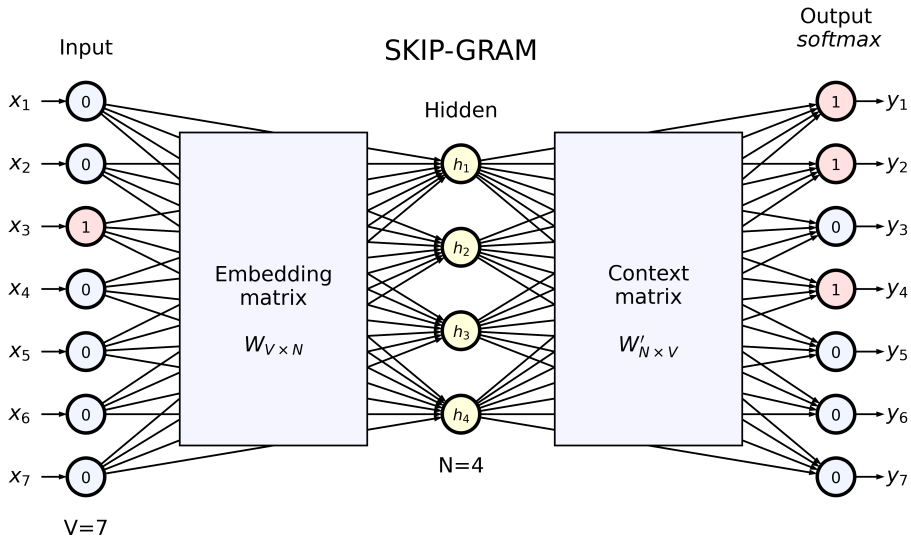
Skip-Gram

The skip-gram model predicts the context words within a specific window given the current word. The input layer of the neural network model uses the current word, and the output layer uses the context words. The hidden layer contains nodes matching the number of dimensions N.

Skip-gram learns by predicting the context for a given target word maximizing:

$$\prod_{t=1}^{T} P(C_t | w_t) \qquad (2)$$

🔶 Example



> "The man who passes the sentence should swing the sword." Ned Stark

Sliding window size $\texttt{ws} = 5$

Window	Target	Context	Skip-Gram
`[The,man,who]`	the	man,who	
`[The,man,who,passes]`	man	the,who,passes	
`[The,man,who,passes,the]`	who	the,man,passes,the	x_3= `who` ; y_1= `the` , y_2= `man` , y_4 = `passes`
`[man,who,passes,the,sentence]`	passes	man,who,the,sentence	
`[sentence,should,swing,the,sword]`	swing	sentence,should,the,sword	
`[should,swing,the,sword]`	the	should,swing,sword	
`[swing,the,sword]`	sword	swing,the	

Disadvantage: A limitation of word embeddings is that possible multiple meanings of a word are conflated into a single representation (unlike Wordnet, which this knowledge is carried in the Wordnet semantic graph)

Embeddings as Features

The bag of words (BoW) model represents text as a set of individual words, disregarding grammar and word order but keeping track of word frequency. It is

commonly used in text processing to convert sentences into numerical features for machine learning.

1. Bag of words can represent similar meanings
2. Skip-gram handles the embeddings not necessarily by similar meaning but via proximity by a nearby word. Focuses on capturing rare words and subtle word relationships, often yielding richer representations for less common words. Performs better in capturing nuanced semantic relationships in sparse or **smaller data**.
3. CBOW predicts the target word based on its surrounding context words, making it more efficient and effective with frequent words but sometimes missing finer word distinctions. Practically, CBOW is faster.

Shortcomings of Bag of Words Method

- It ignores the order of the word, e.g., *this is bad = bad is this*
- It ignores the context of words. Example: "He loved books. Education is best found in books". Two vectors, one for "He loved books", and the other for "Education is best found in books." would be created, and a direct approach could treat these two vectors as **orthogonal** (i.e., independent), losing the relation and context.

Word2Vec

Word2vec techniques use the context of a given word to learn its semantics. Also, Word2vec learns numerical representations of words by looking at the words surrounding a given word.

Imagine an exam where the following sentence is encountered: "Mary is a very stubborn child. Her *pervicacious* nature always gets her in trouble." What does *pervicacious* mean? The phrases surrounding the word of interest are important. In our example, *pervicacious* is surrounded by *stubborn, nature,* and *trouble*. These three words are enough to determine that *pervicacious* in fact means *a state of being stubborn*.

🔶 Gensim

📚 **Gensim** = "Generate Similar" is a topic modeling library to implement Latent Semantic Methods, and it is licensed under 🐐 **GNU LGPLv2.1 license**.

The `Word2Vec` module of `gensim` can generate CBOW and skip-gram models. Here is the 📚 **gensim API**.

Note that `Word2Vec` does not remove stop words because the algorithm relies on the broader context of the sentence to produce high-quality word vectors.

Doc2Vec

Word2vec processes text by *vectorizing* words and generates feature vectors representing words in the corpus. Similarly, Doc2Vec processes the entire variable-length document by *vectorizing* it into fixed-length feature vectors. Doc2Vec uses a similar scheme to Word2Vec by extending the skip-gram or CBOW with an additional document/paragraph vector D. During the training of words, as in Word2Vec, the document vector D is also trained, and thus it represents the document. Here is the 📚 **Doc2Vec API**.

Example

Install gensim: `pip install gensim python-levenshtein` ↩

Let's load nltk dataset named `abc` and use `gensim` to generate word embeddings with CBOW approach.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72

import gensim
from gensim.models import Word2Vec
from nltk.corpus import abc

print(f'gensim version= {gensim.__version__}')
```

```
gensim version= 4.3.3
```

In [2]:
```
%%time

sents = list(abc.sents())

model = Word2Vec(abc.sents(), min_count=2, workers=4)
X = list(model.wv.index_to_key)

# Sanity
print(f'ABC dataset has {len(sents)} sentences')
print(f'gensim model vocabulary has {len(X)} words mapped to N= {model.vec
```

```
ABC dataset has 29059 sentences
gensim model vocabulary has 19484 words mapped to N= 100 dimensions
CPU times: total: 7.47 s
Wall time: 5.57 s
```

In [3]:
```
# The closest words to the word 'science'
science = model.wv.most_similar('science')
print(science)
```

```
[('agriculture', 0.962897002696991), ('Coalition', 0.9451974034309387), ('la
w', 0.9431613683700562), ('management', 0.9408459663391113), ('textile', 0.9
402082562446594), ('biosecurity', 0.9396265149116516), ('general', 0.9383808
374404907), ('descend', 0.9369015693664551), ('bulk', 0.9363269805908203),
('education', 0.9358422756195068)]
```

In [4]:
```
# Distance between computer and science
science12 = model.wv.similarity('science', 'computer')
print(science12)
```

```
0.7866111
```

Let's see another example from Shakespeare's play Hamlet using CBOW and skip-gram methods, respectively.

In [5]:
```
from nltk.corpus import gutenberg

sents = list(gutenberg.sents('shakespeare-hamlet.txt'))
print(sents[0])
```

```
['[', 'The', 'Tragedie', 'of', 'Hamlet', 'by', 'William', 'Shakespeare', '15
99', ']']
```

In [6]:
```
%%time

# CBOW model
model1 = Word2Vec(sents, vector_size=200, sg=0, window=13, min_count=1, ep

# Skip-gram model
model2 = Word2Vec(sents, vector_size=200, sg=1, window=13, min_count=1, ep
```

```
CPU times: total: 4.56 s
Wall time: 1.48 s
```

Let's find out what the model tells us when the context is the names: `['Hamlet', 'Ophelia', 'Ghost']`

In [7]:
```
similarities1b = model1.wv.most_similar(positive=['Hamlet'], topn=20)
similarities1 = model1.wv.most_similar(positive=['Hamlet', 'Ophelia', 'Gho
similarities2 = model2.wv.most_similar(positive=['Hamlet', 'Ophelia', 'Gho
```

In [8]:
```
# Clean the stop words
def filter_words(_sim):
    from nltk.corpus import stopwords
    import re
    stop_words = set(stopwords.words('english'))
    return [(w,p) for w,p in _sim if w.lower() not in stop_words and re.se

similarities1b = filter_words(similarities1b)
```

```
similarities1 = filter_words(similarities1)
similarities2 = filter_words(similarities2)
```

In [9]:
```
for (w1,s1),(w1b,s1b),(w2,s2) in zip(similarities1, similarities1b, simila
    print(f'{w1:16s}{s1:.3f}\t\t{w1b:16s}{s1b:.3f}\t\t{w2:16s}{s2:.3f}')
```

```
Horatio         0.996           Horatio         0.991           Rosincrane
0.869
Thankes         0.993           Ghost           0.991           Manet
0.840
Welcome         0.993           Welcome         0.990           Sister
0.835
Barnardo        0.993           Ophelia         0.990           Marcellus
0.821
Osricke         0.992           Thankes         0.988           Polonius
0.820
goodnight       0.990           goodnight       0.985           Voltumand
0.820
slaine          0.990           Osricke         0.984           Claudius
0.814
Voltumand       0.990           Barnardo        0.984           Gertrude
0.812
Voltemand       0.989           slaine          0.984           Attendant
0.809
Francisco       0.989           Machine         0.984           Osricke
0.807
coniectures     0.989           Physicke        0.983           Queene
0.806
Reynoldo        0.989           wayes           0.983           Farewell
0.804
strongest       0.988           foreknowing     0.982           Drumme
0.798
foreknowing     0.988           Reynoldo        0.982           Laertes
0.797
Fate            0.987           coniectures     0.982           Welcome
0.796
wayes           0.987           stayes          0.982           Guildenstern
e   0.794
Cornelius       0.987           sickly          0.982           fares
0.791
ayde            0.987           Fate            0.981           louingly
0.790
knauery         0.987           Voltumand       0.981           Guildenstern
0.789
grand           0.987           Francisco       0.981           Lucianus
0.786
```

In [10]:
```
# The word embedding matrix
words1a = [w for w,s in similarities1] + ['Hamlet']
X1a = model1.wv[words1a]

words2a = [w for w,s in similarities2] + ['Hamlet']
X2a = model2.wv[words2a]
```

```
# Sanity
print(X1a.shape)
```

```
(21, 200)
```

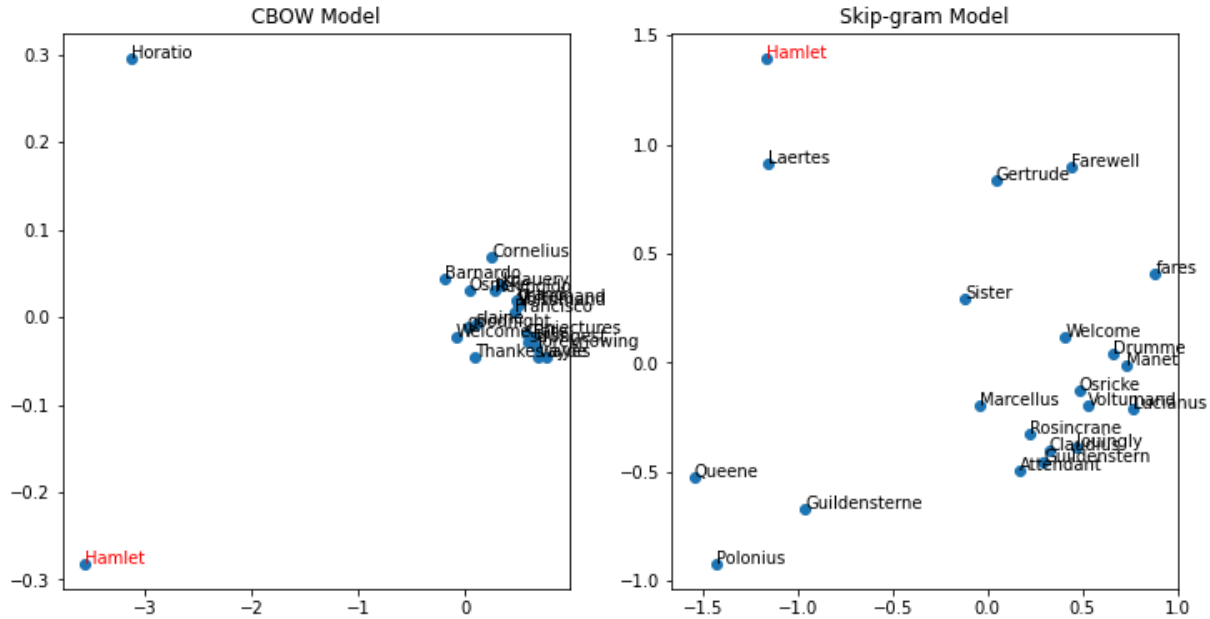
Visualizing the Word Embedding

Using PCA, we can use classical projection methods to reduce the high-dimensional word vectors to two-dimensional plots. The visualizations can provide a qualitative diagnostic for our learned model.

Let's train a projection method on the vectors.

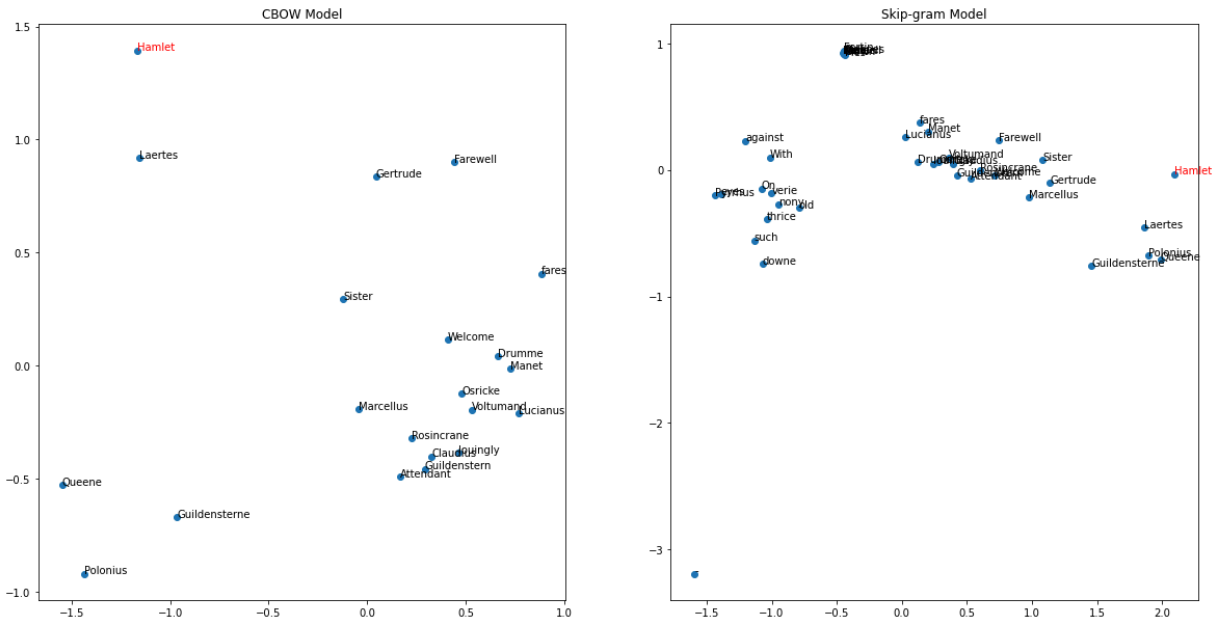
In [11]:
```
from sklearn.decomposition import PCA

pca_model1a = PCA(n_components=2).fit_transform(X1a)
pca_model2a = PCA(n_components=2).fit_transform(X2a)
```

In [12]:
```
def plot_pca(_pca_model, _words, _title):
    plt.scatter(_pca_model[:, 0], _pca_model[:, 1])
    for i, word in enumerate(_words):
        plt.annotate(word, xy=(_pca_model[i, 0], _pca_model[i, 1]), c=('r'
    plt.title(_title)

plt.figure(figsize=(12, 6), dpi=72)

_ = plt.subplot(1, 2, 1)
plot_pca(pca_model1a, words1a, 'CBOW Model')

_ = plt.subplot(1, 2, 2)
plot_pca(pca_model2a, words2a, 'Skip-gram Model')

plt.show()
```



In [13]:
```
dissimilarities1 = model1.wv.most_similar(negative=['Hamlet'], topn=20)
dissimilarities2 = model2.wv.most_similar(negative=['Hamlet'], topn=20)

words1b = ['Hamlet'] + [w for w,s in similarities1] + [w for w,s in dissim

X1b = model1.wv[words1b]

words2b = ['Hamlet'] + [w for w,s in similarities2] + [w for w,s in dissim

X2b = model2.wv[words2b]

pca_model1b = PCA(n_components=2).fit_transform(X1b)
pca_model2b = PCA(n_components=2).fit_transform(X2b)
```

In [14]:
```
plt.figure(figsize=(20, 10), dpi=72)

_ = plt.subplot(1, 2, 1)
plot_pca(pca_model2a, words2a, 'CBOW Model')

_ = plt.subplot(1, 2, 2)
plot_pca(pca_model2b, words2b, 'Skip-gram Model')

plt.show()
```



In [15]:
```
print(model1.wv.most_similar(positive=['Alas', 'poor', 'Yorick', 'Horatio'
```

```
[('ill', 0.99607253074646), ('betweene', 0.9959262013435364), ('yong', 0.995
8122372627258), ('Gertrude', 0.9957283735275269), ('Can', 0.995593309402465
8), ('Oh', 0.9953332543373108), ('warlike', 0.9949916005134583), ('heartil
y', 0.9947196841239929), ('hower', 0.9946835041046143), ('bold', 0.994566321
3729858), ('slaue', 0.9944827556610107), ('looke', 0.9944521188735962), ('sw
eet', 0.994408369064331), ('smiling', 0.9943530559539795), ('Roughly', 0.994
3370819091797), ('cry', 0.9943345785140991), ('hit', 0.9942260980606079),
('strict', 0.9942009449005127), ('Ambassadors', 0.9941619634628296), ('clos
e', 0.9941504597663879)]
```

In [16]:
```
print(model1.wv.most_similar(negative=['Alas', 'poor', 'Yorick', 'Horatio'
```

```
[('Minister', 0.9383090734481812), ('punish', 0.9203029870986938), ('Scourg
e', 0.9091063141822815), ('Satyre', 0.8776348233222961), ('Hiperion', 0.8661
2468957901), ('Lookes', 0.8655646443367004), ('threats', 0.864698708057403
6), ('Button', 0.8639163970947266), ('larger', 0.8138689994812012), ('remast
er', 0.8136327862739563), ('painting', 0.8091925382614136), ('Death', 0.7908
560037612915), ('Came', 0.7763597965240479), ('Happy', 0.7745006680488586),
('satisfaction', 0.765322208404541), ('Abstracts', 0.7631725072860718), ('Va
lentines', 0.7413296699523926), ('Conference', 0.7404723763465881), ('Valent
ine', 0.7338000535964966), ('prison', 0.7271638512611389)]
```

Do you notice any meaningful word from the context `['Alas', 'poor', 'Yorick', 'Horatio']` as in above?

What is that `'O'`?

In [17]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
```

```
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```